

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

**по выполнению курсового проекта
по дисциплине «Технология разработки программного
обеспечения» для направления подготовки 09.03.04 «Программная
инженерия» направленность (профиль) «Разработка и сопровождение
программного обеспечения»**

Ставрополь 2026

ЧАСТЬ 1. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКАЯ

1.1. Введение о трёх траекториях

Курсовой проект по дисциплине «Технология разработки программного обеспечения» предлагает студенту выбор из трёх траекторий выполнения, различающихся по сложности, используемым технологиям и требуемому уровню подготовки.

Траектория А. Full-stack Django (классический подход)

Краткая характеристика:

- Разработка классического веб-приложения с использованием Django Templates
- Серверный рендеринг HTML-страниц
- Сессионная аутентификация (cookies)
- Частичное использование AJAX для динамического обновления контента

Сложность: Низкая

Оценка: «хорошо» (максимум 4)

Траектория Б: Django REST + React SPA (современный подход)

Краткая характеристика:

- Разделение на бэкенд (Django REST API) и фронтенд (React SPA)
- JWT-аутентификация (access + refresh токены)
- Полноценное REST API с CRUD-операциями
- Хранение токенов в localStorage, автоматическое обновление

Сложность: Высокая

Оценка: «отлично» (максимум 5)

Траектория В: React SPA + AJAX + JWT (расширенный подход)

Краткая характеристика:

- Включает все возможности траектории Б
- Расширенная клиентская логика (оптимистичные обновления, кэширование)
- Дополнительный функционал: WebSocket (Django Channels), real-time уведомления
- Продвинутая работа с API (пагинация, фильтрация, поиск на клиенте)

Сложность: Очень высокая

Объём кода: ~5000+ строк

Оценка: «отлично» + автомат на экзамене

1.2. Организация выполнения курсового проекта

Форма организации работы

Индивидуальное выполнение:

- Каждый студент выполняет собственный проект
- Тема выбирается из утверждённого списка
- Полная персональная ответственность за все этапы

Система оценки при условии выполнения всех требований, выбранной траектории.

Траектория	Максимальная оценка	Условие автомата на экзамене
------------	---------------------	------------------------------

А	4 (хорошо)	
Б	5 (отлично)	
В	5 (отлично)	+ автомат при успешной защите

1.3. Общая цель курсового проекта

Закрепление и углубление теоретических знаний, полученных в курсе «Технология разработки программного обеспечения», а также формирование практических навыков проектирования, разработки и документирования веб-приложений с использованием современного технологического стека на основе Django.

1.4. Цели и задачи курсового проекта по траекториям

Траектория А: Full-stack Django

Цель: Разработать полнофункциональное веб-приложение на Django с использованием шаблонов, сессионной аутентификации и базовыми AJAX-запросами.

Задачи:

1. Провести предпроектное обследование и бизнес-анализ предметной области.
2. Разработать модель требований к программному обеспечению.
3. Спроектировать архитектуру веб-приложения на основе Django MTV.
4. Реализовать модели данных и связи между ними.
5. Реализовать систему аутентификации (сессионную).
6. Создать HTML-шаблоны с использованием Bootstrap.
7. Реализовать CRUD-операции для основных сущностей.
8. Добавить AJAX-запросы для динамического обновления контента.

9. Подготовить полный комплект проектной документации.

Траектория Б: Django REST + React SPA

Цель: Разработать современное SPA-приложение с разделением на бэкенд (Django REST API) и фронтенд (React), использующее JWT-аутентификацию.

Задачи:

1. Провести предпроектное обследование и бизнес-анализ предметной области.
2. Разработать модель требований к программному обеспечению.
3. Спроектировать архитектуру клиент-серверного приложения.
4. Реализовать бэкенд на Django REST Framework с REST API.
5. Реализовать систему JWT-аутентификации (access/refresh токены).
6. Настроить разрешения (permissions) для разграничения доступа.
7. Реализовать фронтенд на React с использованием React Router и Axios.
8. Настроить CORS и интеграцию между фронтендом и бэкендом.
9. Обеспечить качество кода через тестирование.
10. Подготовить полный комплект проектной документации.

Траектория В: React SPA + AJAX + JWT (расширенный)

Цель: Разработать расширенное SPA-приложение с продвинутой клиентской логикой, оптимистичными обновлениями, WebSocket и real-time уведомлениями.

Задачи (включая все задачи траектории Б):

- 1-9. Все задачи траектории Б.
10. Реализовать расширенную клиентскую логику (оптимистичные обновления).

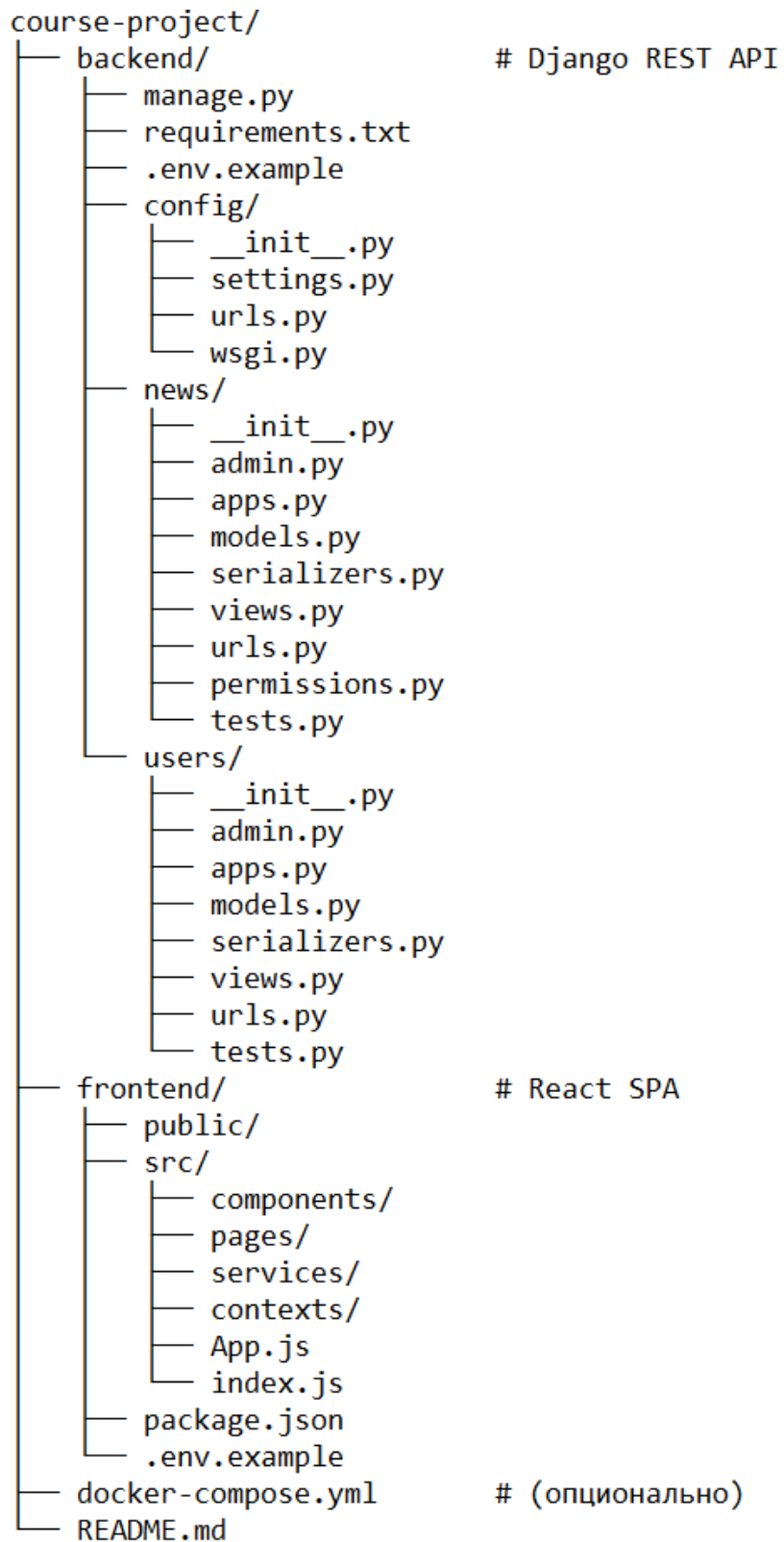
11. Добавить кэширование запросов (React Query / RTK Query).
12. Реализовать пагинацию, фильтрацию и поиск на клиенте.
13. Внедрить WebSocket (Django Channels) для real-time уведомлений.
14. Реализовать систему комментариев с моментальным отображением.
15. Добавить систему избранного с оптимистичным обновлением UI.

1.5. Требования и структура репозитория

Траектория А. единый репозиторий

```
course-project/
├── backend/
│   ├── manage.py
│   ├── requirements.txt
│   ├── .env.example
│   ├── config/
│   │   ├── __init__.py
│   │   ├── settings.py
│   │   ├── urls.py
│   │   └── wsgi.py
│   ├── news/
│   │   ├── __init__.py
│   │   ├── admin.py
│   │   ├── apps.py
│   │   ├── models.py
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── forms.py
│   │   └── tests.py
│   └── users/
│       ├── __init__.py
│       ├── admin.py
│       ├── apps.py
│       ├── models.py
│       ├── views.py
│       ├── urls.py
│       ├── forms.py
│       └── tests.py
├── templates/
│   ├── base.html
│   ├── inc/
│   └── ...
├── static/
├── media/
├── docs/
│   └── images/
│   └── ...
└── README.md
```

Траектория Б: два репозитория (или моно-репозиторий)



Траектория В: расширенная структура

```

course-project/
├── backend/                                # Django REST API + Channels
│   ├── ... (как в траектории Б)
│   ├── consumers.py                       # WebSocket consumers
│   ├── routing.py                         # WebSocket routing
│   └── asgi.py                            # ASGI конфигурация
├── frontend/                              # React SPA + WebSocket
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   │   ├── api.js                    # REST API
│   │   │   └── websocket.js             # WebSocket client
│   │   ├── hooks/
│   │   │   └── useWebSocket.js
│   │   ├── store/                        # Redux Toolkit / RTK Query
│   │   └── ...
│   └── ...
├── docker-compose.yml                     # Django, React, PostgreSQL, Redis
└── README.md

```

ЧАСТЬ 2. ВАРИАНТЫ ЗАДАНИЙ

2.1. Выбор темы проекта

Каждый студент выбирает одну тему из списка. Тема определяет названия сущностей и специфику бизнес-логики.

Таблица вариантов

2.2. Общие требования ко всем вариантам по всем траекториям

№	Категория	Название проекта	Основная модель (Пост)	Модель категории
1	Блоги	Платформа для личных блогов	Post	Topic
2	Спорт	Новости спорта	SportNews	SportType
3	Технологии	IT-новости и обзоры	TechPost	TechCategory
4	Культура	Афиша мероприятий	Event	EventType
5	Образование	Новости университета	UniversityNews	Faculty

6	Наука	Научные статьи	SciencePaper	ScienceField
7	Медицина	Новости медицины	HealthNews	HealthTopic
8	Путешествия	Блог о путешествиях	TravelPost	Country
9	Кино	Новости киноиндустрии	MovieNews	Genre
10	Музыка	Музыкальный блог	MusicPost	MusicGenre
11	Бизнес	Деловые новости	BusinessNews	BusinessSector
12	Политика	Политические новости	PoliticalNews	PoliticalTopic
13	Игры	Новости игровой индустрии	GameNews	GameGenre
14	Кулинария	Кулинарный блог	Recipe	CuisineType

Модели данных

Для каждой темы необходимо создать минимум две связанные модели (категория и основная сущность). В каждой модели минимум 4 поля на таблицу.

Функциональные требования:

Публичная часть (доступна всем):

- Просмотр списка публикаций (с пагинацией)
- Просмотр публикаций по категориям
- Детальный просмотр отдельной публикации
- Поиск по названию/содержанию

Приватная часть (доступна только авторизованным):

- Создание новых публикаций

- Редактирование своих публикаций
- Удаление своих публикаций
- Добавление комментариев

Административная часть (только для администраторов):

- Управление всеми публикациями
- Управление категориями
- Управление пользователями

ЧАСТЬ 3. ЭТАПЫ ВЫПОЛНЕНИЯ ПРОЕКТА

3.0. Сроки и этапы по траекториям

Траектория А (18 недель)

Неделя	Этап	Вес в оценке
1-2	Этап 0: Инициация и бизнес-анализ	5%
3-4	Этап 1: Проектирование требований	10%
5-6	Этап 2: Архитектурное проектирование	10%
7-8	Этап 3: Проектирование базы данных	10%
9-10	Этап 4: Создание и настройка проекта Django	10%
11-12	Этап 5: Реализация моделей и админ-панели	15%
13-14	Этап 6: Реализация представлений и шаблонов	15%
15-16	Этап 7: Аутентификация и AJAX	15%
17-18	Этап 8: Завершение и документация	10%

Траектория Б (18 недель)

Неделя	Этап	Вес в оценке
1-2	Этап 0: Инициация и бизнес-анализ	5%
3-4	Этап 1: Проектирование требований	10%
5-6	Этап 2: Архитектурное проектирование	10%
7-8	Этап 3: Проектирование базы данных	10%
9-10	Этап 4: Создание и настройка проекта Django	10%
11-12	Этап 5: Реализация моделей и админ-панели	10%
13-14	Этап 6: Реализация API и JWT-аутентификации	15%
15-16	Этап 7: Разработка React-приложения	15%
17-18	Этап 8: Завершение и документация	15%

Траектория В (18 недель)

Неделя	Этап	Вес в оценке
1-2	Этап 0: Инициация и бизнес-анализ	5%
3-4	Этап 1: Проектирование требований	10%
5-6	Этап 2: Архитектурное проектирование	10%
7-8	Этап 3: Проектирование базы данных	10%
9-10	Этап 4: Создание и настройка проекта Django	10%
11-12	Этап 5: Реализация моделей и админ-панели	10%
13-14	Этап 6: Реализация API и JWT-аутентификации	10%
15-16	Этап 7: Разработка React-приложения + WebSocket	15%
17-18	Этап 8: Завершение и документация	20%

3.1. Этап 0. Инициация и бизнес-анализ (Недели 1-2)

Задачи этапа (общие для всех траекторий)

1. Выбор темы проекта из списка.
2. Анализ предметной области.
3. Определение ключевых сущностей и их атрибутов.
4. Создание бизнес-модели.

Результаты этапа

Артефакт	Описание	Формат
Паспорт проекта	Краткое изложение (1 стр.)	Документ
Диаграмма бизнес-контекста	IDEF0 A-0	UML-диаграмма
BUC-диаграмма	Бизнес-прецеденты	UML-диаграмма
Бизнес-гlossарий	10+ терминов	Таблица
Матрица стейкхолдеров	Анализ заинтересованных лиц	Таблица
SWOT-анализ	Анализ текущего процесса	Таблица

3.2. Этап 1. Проектирование требований (Недели 3-4)

Задачи этапа (общие для всех траекторий)

1. Разработка Use Case диаграммы.
2. Создание доменной модели.
3. Детализация функциональных требований.

Результаты этапа

Артефакт	Описание	Формат
----------	----------	--------

Паспорт проекта	Краткое изложение (1 стр.)	Документ
Диаграмма бизнес-контекста	IDEF0 A-0	UML-диаграмма
BUC-диаграмма	Бизнес-прецеденты	UML-диаграмма
Бизнес-гlossарий	10+ терминов	Таблица
Матрица стейкхолдеров	Анализ заинтересованных лиц	Таблица
SWOT-анализ	Анализ текущего процесса	Таблица

3.3. Этап 2. Архитектурное проектирование (Недели 5-6)

В курсовом проекте должно быть как минимум одно приложение предметной области и приложение users.

Приложение users - это модуль Django, отвечающий за управление пользователями, аутентификацию и разграничение доступа в веб-приложении.

Назначение приложение users

1. Регистрация - создание новых учётных записей пользователей
2. Аутентификация - проверка подлинности пользователя (вход/выход)
3. Авторизация - определение прав доступа к различным частям приложения
4. Управление профилем - просмотр и редактирование личной информации

2. Модели данных

В приложении users должны содержаться модели для хранения информации о пользователях:

User - расширенная модель пользователя (наследуется от AbstractUser или AbstractBaseUser)

Profile - Дополнительная информация о пользователе (аватар, биография, телефон)

3. API-эндпоинты (примеры)

Приложение users предоставляет API для работы с пользователями:

Эндпоинт	Метод	Назначение
/api/auth/register/	POST	Регистрация нового пользователя
/api/auth/login/	POST	Получение JWT-токенов (access и refresh)
/api/auth/token/refresh/	POST	Обновление access-токена
/api/auth/profile/	GET	Получение данных профиля
/api/auth/profile/	PUT/PATCH	Обновление профиля

4. Разграничение доступа

Приложение users должно обеспечивать три уровня доступа:

Уровень	Роль	Возможности
1	Гость (неавторизован)	Только просмотр публичного контента
2	Пользователь (авторизован)	Просмотр + создание/редактирование своих данных
3	Администратор (is_staff)	Полный доступ к управлению приложением

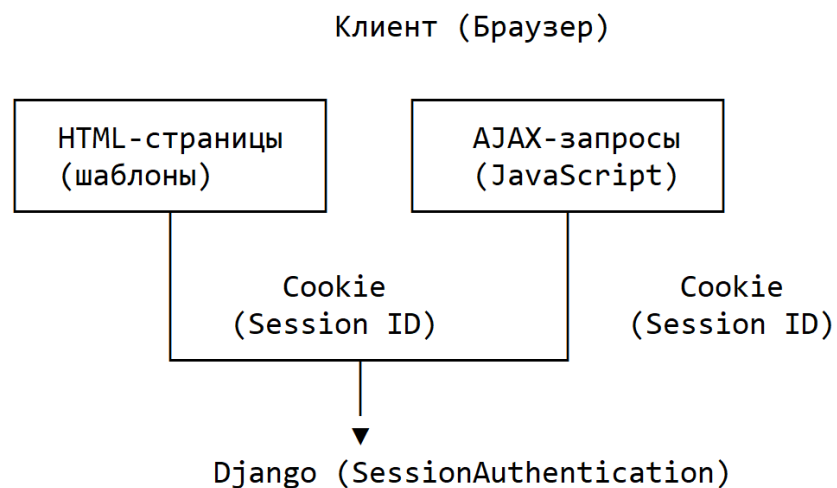
5. Логическая изоляция

Выделение логики управления пользователями в отдельное приложение users обеспечивает:

Преимущество	Описание
Чистая архитектура	Разделение ответственности - основное приложение (news) занимается контентом, а users - пользователями
Переиспользование	При необходимости приложение users можно использовать в других проектах
Упрощение поддержки	Изменения в аутентификации не затрагивают основную бизнес-логику
Безопасность	Централизованное управление доступом и правами

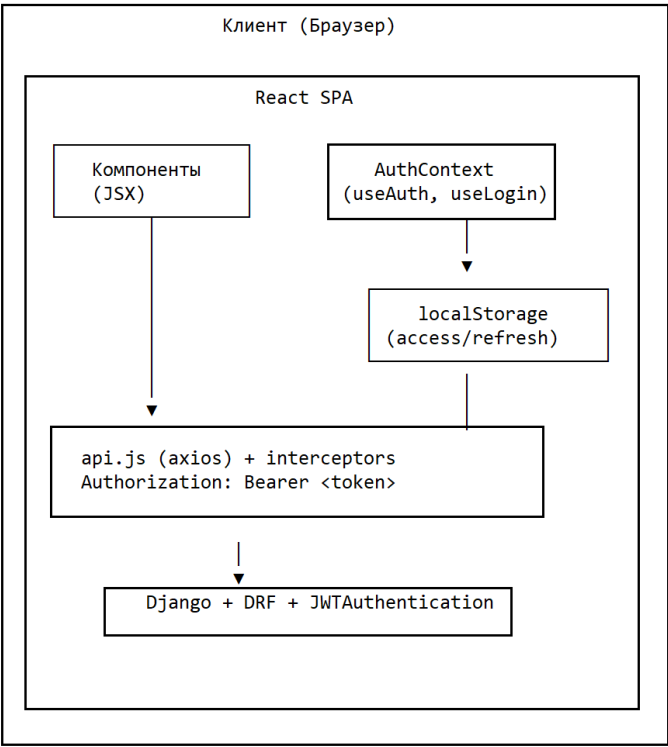
Траектория А: Архитектура Django MTV

Общая схема взаимодействия:

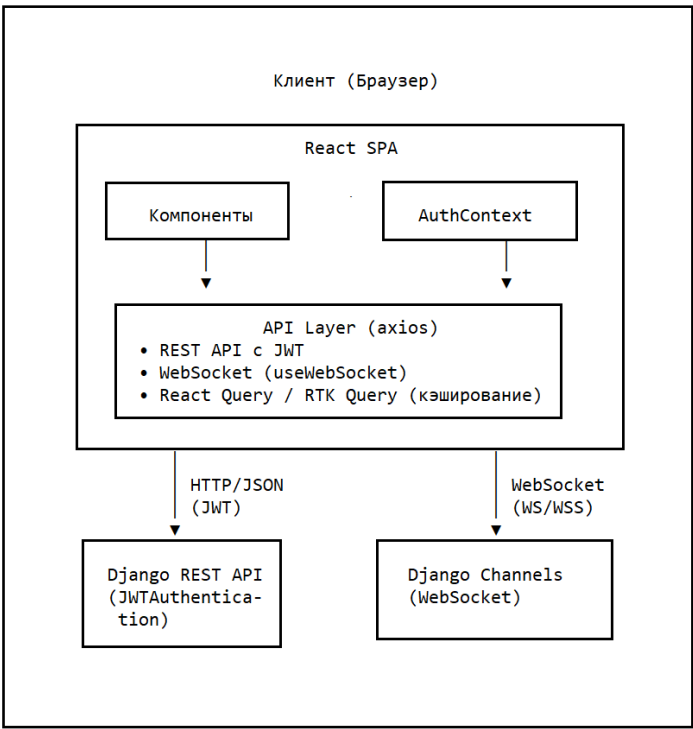


Траектория Б: Архитектура Django REST + React SPA

Общая схема взаимодействия:



Траектория В: Архитектура React SPA + WebSocket



Общая схема взаимодействия:

Результаты этапа (общие для всех траекторий)

Артефакт	Описание	Формат
----------	----------	--------

Диаграмма компонентов	Структура системы	UML-диаграмма
Спецификация API	Список эндпоинтов	Документ / Markdown
Схема аутентификации	Поток получения токенов	Диаграмма

3.4. Этап 3. Проектирование базы данных (Недели 7-8)

Требования к БД (общие для всех траекторий)

- Нормализация до 3 нормальной формы
- Чёткое определение первичных и внешних ключей
- Индексы для часто запрашиваемых полей
- Ограничения целостности (NOT NULL, UNIQUE, CHECK)

Пример описания таблиц атрибутов моделей приложений

Модель User (приложение users)

Поле	Тип	Описание	Ограничения
id	Integer	Первичный ключ	PK, автоинкремент
username	CharField	Имя пользователя	UNIQUE, NOT NULL
email	EmailField	Email	UNIQUE
password_hash	CharField	Хеш пароля	NOT NULL
first_name	CharField	Имя	
last_name	CharField	Фамилия	
is_staff	BooleanField	Администратор	default=False
is_active	BooleanField	Активен	default=True
bio	TextField	О себе	blank=True
avatar	ImageField	Аватар	blank=True

phone	CharField	Телефон	blank=True
-------	-----------	---------	------------

Модель Category (приложение news)

Поле	Тип	Описание	Ограничения
id	Integer	Первичный ключ	PK, автоинкремент
title	CharField	Название категории	NOT NULL, db_index=True

Модель Article (приложение news)

Поле	Тип	Описание	Ограничения
id	Integer	Первичный ключ	PK, автоинкремент
title	CharField	Заголовок	NOT NULL
content	TextField	Содержание	NOT NULL
created_at	DateTimeField	Дата создания	auto_now_add=True
updated_at	DateTimeField	Дата обновления	auto_now=True
photo	ImageField	Изображение	blank=True
is_published	BooleanField	Опубликовано	default=True
views	IntegerField	Просмотры	default=0
category_id	ForeignKey	Категория	FK: Category, on_delete=PROTECT
author_id	ForeignKey	Автор	FK: User, on_delete=SET_NULL

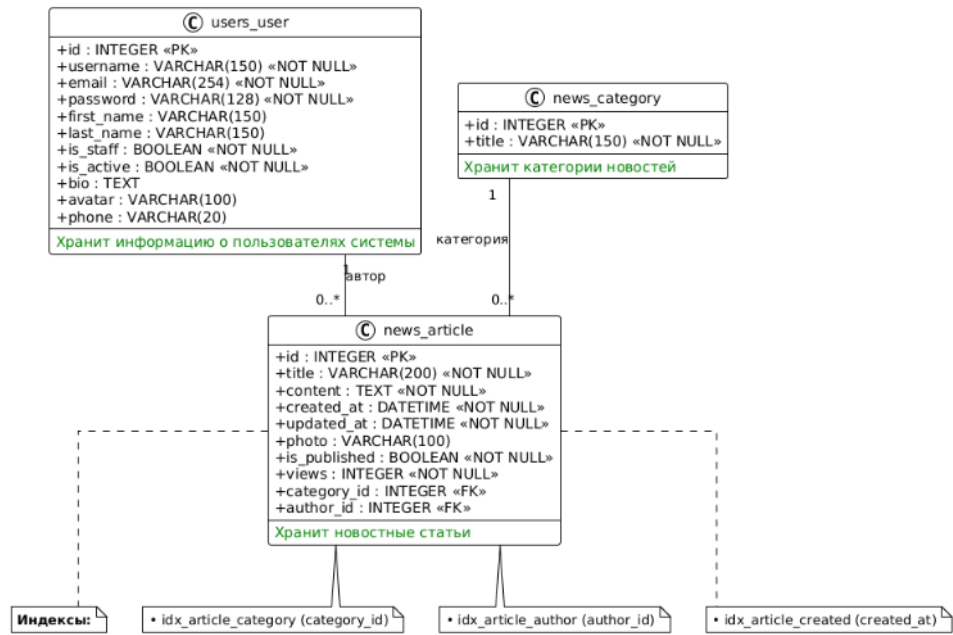
Связи между таблицами

Связь	Тип	Описание
Category → Article	один ко многим	Одна категория может содержать много статей

User → Article	один ко многим	Один пользователь может быть автором многих статей
----------------	----------------	--

ER-диаграмма (общая для всех траекторий)

ER-диаграмма: User, Category, Article



Результаты этапа

Артефакт	Описание	Формат	Примечание
ER-диаграмма	Логическая модель данных	UML-диаграмма	
DDL-скрипты	SQL-скрипты создания таблиц	SQL	Допустимо взять из СУБД после миграции моделей
Описание связей	ForeignKey, on_delete	Документ	

3.5. Этап 4. Создание и настройка проекта Django

3.5.1. Создание виртуального окружения (общее для всех траекторий)

```
python -m venv venv  
venv\Scripts\activate
```

3.5.2. Установка необходимых пакетов

Траектория А

```
pip install django pillow  
pip install python-dotenv
```

Траектория Б

```
pip install django djangorestframework djangorestframework-simplejwt  
pip install pillow django-cors-headers psycopg2-binary  
pip install python-dotenv
```

Траектория В

```
pip install django djangorestframework djangorestframework-simplejwt  
pip install pillow django-cors-headers psycopg2-binary channels  
pip install python-dotenv
```

3.5.3. Создание проекта и приложений (общее для всех траекторий)

```
django-admin startproject config .  
python manage.py startapp news  
python manage.py startapp users
```

3.5.4. Настройка settings.py

Траектория А: Файл config/settings.py

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-m5aew+)1+5w1c09$h=%ty_o)7d3d%4)-_mcl!gxk_0n6g@8l3t'

DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'news.apps.NewsConfig',
    'users.apps.UsersConfig',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'configA.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'configA.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = [
```

```

        {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',},
        {'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',},
        {'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',},
        {'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',},
    ]

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles'

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
AUTH_USER_MODEL = 'users.User'

```

Траектория Б: Файл config/settings.py (дополнительно к траектории А)

Прежде установите `pip install python-dotenv` и `pip install django-cors-headers`, если не установили ранее

```

import os
from pathlib import Path
from datetime import timedelta
from dotenv import load_dotenv

# Загрузка переменных окружения из файла .env
load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-m5aew+)1+5w1c09$h=%ty_o)7d3d%4)-_mcl!gxk_0n6g@8l3t'

DEBUG = True

ALLOWED_HOSTS = []

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'Third-party apps',
    'rest_framework',
    'rest_framework_simplejwt',
    'corsheaders',

```

```

    'news.apps.NewsConfig',
    'users.apps.UsersConfig',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'configA.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'configA.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',},
    {'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator',},
    {'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator',},
    {'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator',},
]

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles'

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

```

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
AUTH_USER_MODEL = 'users.User'
```

Траектория В

```
from pathlib import Path
import os
from datetime import timedelta
from dotenv import load_dotenv
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from django.urls import path

load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')

SECRET_KEY = 'django-insecure-t9ya_4x&4cu#d@5e&2$n^z1^-
qql*#5igw+s@#o08y3l@cdlx'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'rest_framework_simplejwt',
    'corsheaders',
    'channels', # Django Channels для WebSocket
    'news.apps.NewsConfig',
    'users.apps.UsersConfig',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'configC.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
```



```

        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

# ASGI application (for WebSocket support) асинхронная поддержка
ASGI_APPLICATION = 'config.asgi.application'
WSGI_APPLICATION = 'configC.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
     'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',},
    {'NAME':
     'django.contrib.auth.password_validation.MinimumLengthValidator',},
    {'NAME':
     'django.contrib.auth.password_validation.CommonPasswordValidator',},
    {'NAME':
     'django.contrib.auth.password_validation.NumericPasswordValidator',},
]

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]

MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

# Default primary key field type
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
# Кастомные пользовательские модели
AUTH_USER_MODEL = 'users.User'

# DJANGO REST FRAMEWORK (DRF) настройки
REST_FRAMEWORK = {
    # Authentication classes
    'DEFAULT_AUTHENTICATION_CLASSES': (

```

```

        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),

    # Permission classes
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticatedOrReadOnly',
    ),

    # Pagination
    'DEFAULT_PAGINATION_CLASS':
'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,

    # Filtering
    'DEFAULT_FILTER_BACKENDS': (
        'django_filters.rest_framework.DjangoFilterBackend',
    ),
}

# простые JWT настройки
SIMPLE_JWT = {
    # Token lifetimes
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),

    # Token rotation
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,

    # Algorithm
    'ALGORITHM': 'HS256',
    'SIGNING_KEY': SECRET_KEY,
    'VERIFYING_KEY': None,

    # Headers
    'AUTH_HEADER_TYPES': ('Bearer',),
    'AUTH_HEADER_NAME': 'HTTP_AUTHORIZATION',

    # User identification
    'USER_ID_FIELD': 'id',
    'USER_ID_CLAIM': 'user_id',
}

# CORS SETTINGS
# Разрешённые источники для CORS
CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
    "http://127.0.0.1:3000",
]

# Разрешённые методы
CORS_ALLOW_METHODS = [
    'DELETE',
    'GET',
    'OPTIONS',
    'PATCH',
    'POST',
    'PUT',
]

# Разрешённые заголовки
CORS_ALLOW_HEADERS = [
    'accept',

```

```

    'accept-encoding',
    'authorization',
    'content-type',
    'dnt',
    'origin',
    'user-agent',
    'x-csrfToken',
    'x-requested-with',
]

# Разрешить отправку credentials (cookies, authorization headers)
CORS_ALLOW_CREDENTIALS = True

# CHANNELS (WEBSOCKET) настройки
# Channel layers configuration
CHANNEL_LAYERS = {
    'default': {
        # Development: in-memory layer (no Redis required)
        'BACKEND': 'channels.layers.InMemoryChannelLayer',

        # Production: Redis channel layer (uncomment for deployment)
        # 'BACKEND': 'channels_redis.core.RedisChannelLayer',
        # 'CONFIG': {
        #     "hosts": [("127.0.0.1", 6379)],
        # },
    },
}

```

3.5.5. Настройка URL-маршрутов

Траектория А: config/urls.py

```

from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('news.urls')),
    path('users/', include('users.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

```

Траектория Б: config/urls.py

```

from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('news.urls')),
]

```

```

        path('api/auth/', include('users.urls')),
    ]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

```

Траектория В

config/urls.py (аналогично траектории Б)

```

from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

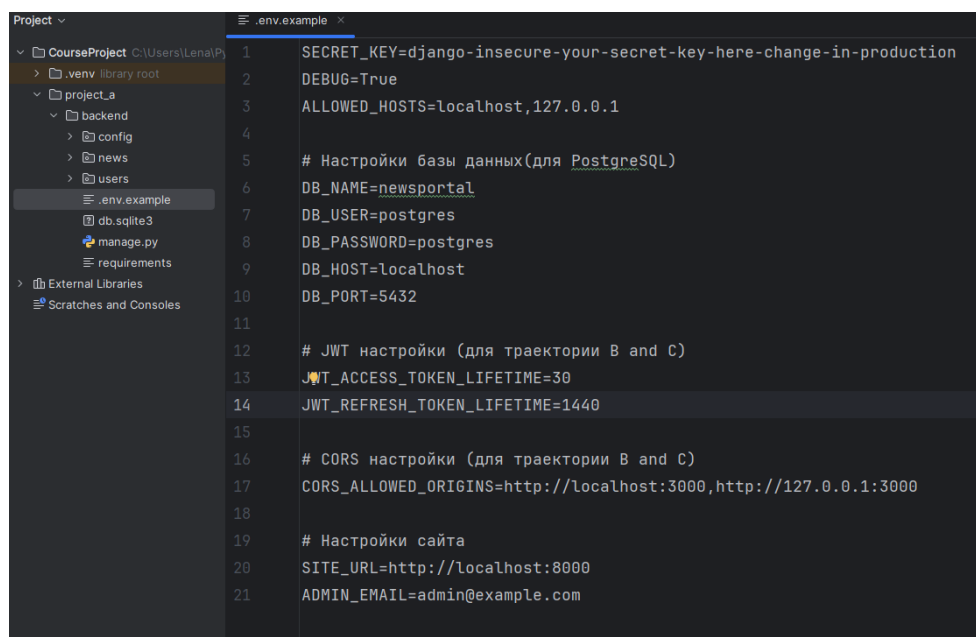
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('news.urls')),
    path('api/auth/', include('users.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

```

Дополнительно: WebSocket маршруты будут в routing.py

3.5.6. Файл переменных окружения .env.example



3.6. Этап 5. Реализация моделей и админ-панели

3.6.1. Модель пользователя (общая для всех траекторий)

Файл: users/models.py

```
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    bio = models.TextField(verbose_name='О себе', blank=True)
    avatar = models.ImageField(upload_to='avatars/%Y/%m/%d',
                               verbose_name='Аватар', blank=True)
    phone = models.CharField(max_length=20, verbose_name='Телефон',
                             blank=True)

    class Meta:
        verbose_name = 'Пользователь'
        verbose_name_plural = 'Пользователи'

    def __str__(self):
        return self.username
```

3.6.2. Настройка пользовательской модели (общая)

Файл: config/settings.py

```
# Кастомные пользовательские модели
AUTH_USER_MODEL = 'users.User'
```

3.6.3. Модели новостного приложения (общие)

Файл: news/models.py

```
from django.db import models
from django.conf import settings

class Category(models.Model):
    title = models.CharField(max_length=150, db_index=True,
                             verbose_name='Категория')

    def __str__(self):
        return self.title

    class Meta:
        verbose_name = 'Категория'
        verbose_name_plural = 'Категории'
        ordering = ['title']
```

```

class Article(models.Model):
    title = models.CharField(max_length=200, verbose_name='Заголовок')
    content = models.TextField(verbose_name='Содержание')
    created_at = models.DateTimeField(auto_now_add=True, verbose_name='Дата
создания')
    updated_at = models.DateTimeField(auto_now=True, verbose_name='Дата
обновления')
    photo = models.ImageField(upload_to='photos/%Y/%m/%d',
verbose_name='Фото', blank=True)
    is_published = models.BooleanField(default=True,
verbose_name='Опубликовано')
    category = models.ForeignKey(Category, on_delete=models.PROTECT,
verbose_name='Категория')
    views = models.IntegerField(default=0, verbose_name='Просмотры')
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        verbose_name='Автор'
    )

    def __str__(self):
        return self.title

class Meta:
    verbose_name = 'Новость'
    verbose_name_plural = 'Новости'
    ordering = ['-created_at']

```

3.6.4. Миграции (общие)

Предварительно настройте файлы `urls.py` и `views.py` в собственном приложении и в `users`

```
python manage.py makemigrations
```

```
python manage.py migrate
```

3.6.5. Настройка админ-панели (общая)

Файл: `news/admin.py`

```

from django.contrib import admin
from .models import Category, Article

@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ('id', 'title')
    list_display_links = ('id', 'title')
    search_fields = ('title',)

@admin.register(Article)
class ArticleAdmin(admin.ModelAdmin):

```

```
list_display = ('id', 'title', 'category', 'author', 'created_at',
'is_published', 'views')
list_display_links = ('id', 'title')
search_fields = ('title', 'content')
list_editable = ('is_published',)
list_filter = ('is_published', 'category', 'author')
readonly_fields = ('views',)
```

Файл: users/admin.py

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .models import User

@admin.register(User)
class CustomUserAdmin(UserAdmin):
    list_display = ('username', 'email', 'first_name', 'last_name',
'is_staff')
    fieldsets = UserAdmin.fieldsets + (
        ('Дополнительная информация', {'fields': ('bio', 'avatar',
'phone')}),
    )
```

3.7. Этап 6. Реализация представлений и API

3.7.1. Траектория А: Представления (views.py) и шаблоны

Файл: news/views.py

```
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from .models import Article, Category
from .forms import ArticleForm

def home(request):
    articles =
Article.objects.filter(is_published=True).select_related('category',
'author')
    categories = Category.objects.all()
    context = {
        'articles': articles,
        'categories': categories,
        'title': 'Главная страница'
    }
    return render(request, 'news/home.html', context)

def category_detail(request, pk):
    category = get_object_or_404(Category, pk=pk)
    articles = Article.objects.filter(category=category, is_published=True)
    categories = Category.objects.all()
    context = {
        'category': category,
        'articles': articles,
        'categories': categories,
        'title': category.title
    }
    }
```

```

        return render(request, 'news/category.html', context)

def post_detail(request, pk):
    article = get_object_or_404(Article, pk=pk, is_published=True)
    article.views += 1
    article.save(update_fields=['views'])
    categories = Category.objects.all()
    context = {
        'article': article,
        'categories': categories,
        'title': article.title
    }
    return render(request, 'news/post_detail.html', context)

@login_required
def create_article(request):
    if request.method == 'POST':
        form = ArticleForm(request.POST, request.FILES)
        if form.is_valid():
            article = form.save(commit=False)
            article.author = request.user
            article.save()
            return redirect('post_detail', pk=article.pk)
    else:
        form = ArticleForm()
    return render(request, 'news/create.html', {'form': form})

@login_required
def edit_article(request, pk):
    article = get_object_or_404(Article, pk=pk, author=request.user)
    if request.method == 'POST':
        form = ArticleForm(request.POST, request.FILES, instance=article)
        if form.is_valid():
            form.save()
            return redirect('post_detail', pk=article.pk)
    else:
        form = ArticleForm(instance=article)
    return render(request, 'news/edit.html', {'form': form, 'article': article})

@login_required
def delete_article(request, pk):
    article = get_object_or_404(Article, pk=pk, author=request.user)
    if request.method == 'POST':
        article.delete()
        return redirect('home')
    return render(request, 'news/delete.html', {'article': article})

# AJAX-запрос для комментариев
@login_required
def add_comment_ajax(request):
    if request.method == 'POST':
        article_id = request.POST.get('article_id')
        text = request.POST.get('text')
        article = get_object_or_404(Article, pk=article_id)
        comment = Comment.objects.create(
            article=article,
            author=request.user,
            text=text
        )
    return JsonResponse({
        'status': 'success',
        'comment': {
            'id': comment.id,

```



```

        'text': comment.text,
        'author': comment.author.username,
        'created_at': comment.created_at.strftime('%d.%m.%Y %H:%M')
    }
})
return JsonResponse({'status': 'error'}, status=400)

```

Файл: news/forms.py

```

from django import forms
from .models import Article

class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = ['title', 'content', 'category', 'photo', 'is_published']
        widgets = {
            'content': forms.Textarea(attrs={'rows': 10, 'class': 'form-control'}),
            'title': forms.TextInput(attrs={'class': 'form-control'}),
            'category': forms.Select(attrs={'class': 'form-control'}),
            'photo': forms.FileInput(attrs={'class': 'form-control'}),
            'is_published': forms.CheckboxInput(attrs={'class': 'form-check-input'}),
        }

```

Файл: news/urls.py (траектория А)

```

from django.urls import path
from . import views

app_name = 'news'

urlpatterns = [
    path('', views.home, name='home'),
    path('category/<int:pk>/', views.category_detail,
name='category_detail'),
    path('post/<int:pk>/', views.post_detail, name='post_detail'),
    path('create/', views.create_article, name='create'),
    path('edit/<int:pk>/', views.edit_article, name='edit'),
    path('delete/<int:pk>/', views.delete_article, name='delete'),
    path('api/add_comment/', views.add_comment_ajax,
name='add_comment_ajax'),
]

```

3.7.2. Траектория Б: Сериализаторы и ViewSet'ы

Файл: news/serializers.py

```

from rest_framework import serializers
from .models import Category, Article

class CategorySerializer(serializers.ModelSerializer):

```

```

        article_count = serializers.IntegerField(source='article_set.count',
read_only=True)

        class Meta:
            model = Category
            fields = ['id', 'title', 'article_count']

class ArticleListSerializer(serializers.ModelSerializer):
    category_title = serializers.CharField(source='category.title',
read_only=True)
    author_name = serializers.CharField(source='author.username',
read_only=True)

    class Meta:
        model = Article
        fields = ['id', 'title', 'created_at', 'views', 'category_title',
'author_name', 'photo']

class ArticleDetailSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)
    author = serializers.StringRelatedField(read_only=True)

    class Meta:
        model = Article
        fields = '__all__'

class ArticleCreateUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = ['title', 'content', 'category', 'photo', 'is_published']

```

Файл: news/permissions.py

```

from rest_framework import permissions

class IsAuthorOrReadOnly(permissions.BasePermission):
    """Разрешение, позволяющее редактировать статью только её автору."""

    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:
            return True
        return obj.author == request.user

class IsAdminOrReadOnly(permissions.BasePermission):
    """Разрешение, позволяющее изменять данные только администраторам."""

    def has_permission(self, request, view):
        if request.method in permissions.SAFE_METHODS:
            return True
        return request.user and request.user.is_staff

```

Файл: news/views.py (траектория Б)

```

from rest_framework import viewsets, permissions, filters
from rest_framework.decorators import action
from rest_framework.response import Response

```

```

from .models import Category, Article
from .serializers import CategorySerializer, ArticleListSerializer,
ArticleDetailSerializer, \
    ArticleCreateUpdateSerializer
from .permissions import IsAuthorOrReadOnly

class CategoryViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Category.objects.all()
    serializer_class = CategorySerializer
    permission_classes = [permissions.AllowAny]

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.filter(is_published=True)
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ['title', 'content']
    ordering_fields = ['created_at', 'views']

    def get_serializer_class(self):
        if self.action == 'list':
            return ArticleListSerializer
        elif self.action in ['create', 'update', 'partial_update']:
            return ArticleCreateUpdateSerializer
        return ArticleDetailSerializer

    def get_permissions(self):
        if self.action in ['update', 'partial_update', 'destroy']:
            return [IsAuthorOrReadOnly()]
        elif self.action == 'create':
            return [permissions.IsAuthenticated()]
        return [permissions.AllowAny()]

    def perform_create(self, serializer):
        serializer.save(author=self.request.user)

    @action(detail=True, methods=['post'])
    def increment_views(self, request, pk=None):
        article = self.get_object()
        article.views += 1
        article.save(update_fields=['views'])
        return Response({'views': article.views})

    @action(detail=False, methods=['get'],
permission_classes=[permissions.IsAuthenticated])
    def my_articles(self, request):
        articles = Article.objects.filter(author=request.user)
        serializer = self.get_serializer(articles, many=True)
        return Response(serializer.data)

```

Файл: news/urls.py (траектория Б)

```

from django.urls import path, include
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register(r'categories', views.CategoryViewSet)
router.register(r'articles', views.ArticleViewSet)

urlpatterns = [
    path('', include(router.urls)),
]

```

3.7.3. Трассировка В: Дополнительные возможности

WebSocket Consumer (В)

Файл: config/asgi.py

```
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from django.urls import path

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')

from news import consumers

application = ProtocolTypeRouter({
    "http": get_asgi_application(),
    "websocket": AuthMiddlewareStack(
        URLRouter([
            path("ws/articles/<int:article_id>/",
consumers.ArticleConsumer.as_asgi()),
        ])
    ),
})
```

Файл: news/consumers.py

```
import json
from channels.generic.websocket import AsyncWebsocketConsumer
from channels.db import database_sync_to_async
from .models import Article

class ArticleConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.article_id = self.scope['url_route']['kwargs']['article_id']
        self.room_group_name = f'article_{self.article_id}'

        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name
        )
        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(
            self.room_group_name,
            self.channel_name
        )

    async def receive(self, text_data):
        data = json.loads(text_data)

        if data['type'] == 'comment':
            comment = await self.save_comment(data['text'], data['user_id'])

            await self.channel_layer.group_send(
```

```

        self.room_group_name,
        {
            'type': 'new_comment',
            'comment': comment
        }
    )

    async def new_comment(self, event):
        await self.send(text_data=json.dumps({
            'type': 'comment',
            'comment': event['comment']
        }))

    @database_sync_to_async
    def save_comment(self, text, user_id):
        from django.contrib.auth import get_user_model
        from .models import Comment

        User = get_user_model()
        article = Article.objects.get(pk=self.article_id)
        user = User.objects.get(pk=user_id)
        comment = Comment.objects.create(
            article=article,
            user=user,
            text=text
        )
        return {
            'id': comment.id,
            'text': comment.text,
            'user': user.username,
            'created_at': comment.created_at.strftime('%d.%m.%Y %H:%M')
        }

```

Файл: news/routing.py (WebSocket маршруты)

```

from django.urls import path
from . import consumers

websocket_urlpatterns = [
    path('ws/articles/<int:article_id>/',
        consumers.ArticleConsumer.as_asgi()),
]

```

3.8. Этап 7. Аутентификация

3.8.1. Траектория А: Сессионная аутентификация

Файл: users/forms.py

```

from django import forms
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from .models import User

class RegisterForm(UserCreationForm):

```

```

username = forms.CharField(max_length=100, label='Имя пользователя')
email = forms.EmailField(label='Email')
password1 = forms.CharField(widget=forms.PasswordInput, label='Пароль')
password2 = forms.CharField(widget=forms.PasswordInput,
label='Подтверждение')

class Meta:
    model = User
    fields = ['username', 'email', 'password1', 'password2']

class LoginForm(AuthenticationForm):
    username = forms.CharField(max_length=100, label='Имя пользователя')
    password = forms.CharField(widget=forms.PasswordInput, label='Пароль')

class ProfileUpdateForm(forms.ModelForm):
    class Meta:
        model = User
        fields = ['first_name', 'last_name', 'email', 'bio', 'avatar',
'phone']

```

Файл: users/views.py

```

from django.shortcuts import render, redirect
from django.contrib import messages
from django.contrib.auth import login, logout, authenticate
from django.contrib.auth.decorators import login_required
from .forms import RegisterForm, LoginForm, ProfileUpdateForm

def register(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, 'Регистрация прошла успешно!')
            return redirect('news:home')
    else:
        form = RegisterForm()
    return render(request, 'users/register.html', {'form': form})

def user_login(request):
    if request.method == 'POST':
        form = LoginForm(data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f'Добро пожаловать, {username}!')
                return redirect('news:home')
    else:
        form = LoginForm()
    return render(request, 'users/login.html', {'form': form})

def user_logout(request):
    logout(request)
    messages.info(request, 'Вы вышли из системы')
    return redirect('news:home')

@login_required

```

```
def profile(request):
    if request.method == 'POST':
        form = ProfileUpdateForm(request.POST, request.FILES,
instance=request.user)
        if form.is_valid():
            form.save()
            messages.success(request, 'Профиль успешно обновлён!')
            return redirect('profile')
    else:
        form = ProfileUpdateForm(instance=request.user)
    return render(request, 'users/profile.html', {'form': form})
```

Файл: users/urls.py (траектория А)

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register, name='register'),
    path('login/', views.user_login, name='login'),
    path('logout/', views.user_logout, name='logout'),
    path('profile/', views.profile, name='profile'),
]
```

3.8.2. Траектории Б и В: JWT-аутентификация

Файл: users/serializers.py

```
from rest_framework import serializers
from django.contrib.auth import get_user_model
from django.contrib.auth.password_validation import validate_password

User = get_user_model()

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'first_name', 'last_name',
'bio', 'avatar', 'phone']

class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, required=True,
validators=[validate_password])
    password2 = serializers.CharField(write_only=True, required=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password', 'password2', 'first_name',
'last_name']

    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError({"password": "Пароли не
совпадают"})
        return attrs

    def create(self, validated_data):
        validated_data.pop('password2')
```

```

        user = User.objects.create_user(**validated_data)
        return user

class ProfileUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['first_name', 'last_name', 'email', 'bio', 'avatar',
'phone']

```

Файл: users/views.py (траектории Б и В)

```

from rest_framework import generics, permissions
from .serializers import UserSerializer, RegisterSerializer,
ProfileUpdateSerializer

class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [permissions.AllowAny]

class ProfileView(generics.RetrieveUpdateAPIView):
    serializer_class = ProfileUpdateSerializer
    permission_classes = [permissions.IsAuthenticated]

    def get_object(self):
        return self.request.user

```

Файл: users/urls.py (траектории Б и В)

```

from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView,
TokenRefreshView
from . import views

urlpatterns = [
    path('register/', views.RegisterView.as_view(), name='register'),
    path('login/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    path('profile/', views.ProfileView.as_view(), name='profile'),
]

```

3.9. Этап 8. Завершение и документация

Результаты этапа (общие для всех траекторий)

Артефакт	Описание	Формат
WBS	Иерархическая структура работ	Диаграмма
Диаграмма Ганта	Календарный план	Диаграмма

Техническое задание	Полный документ требований	Документ
Руководство пользователя	Инструкция по работе	Документ
Руководство администратора	Инструкция по развёртыванию	Документ
Пояснительная записка	Полный отчёт о проекте	Документ
Презентация	Защита проекта	PDF/PPT

Требования к Git-статистике

В файл README.md обязательно включить раздел «Статистика разработки»:

Статистика разработки

Метрики Git

- Всего коммитов: 47
- Период: 01.04.2026 - 10.06.2026
- Средняя частота: 2.9 коммита/неделю

График активности

![Активность коммитов](docs/images/git-commit-activity.png)

Тепловая карта

![Распределение по времени](docs/images/git-punch-card.png)

ЧАСТЬ 4. КРИТЕРИИ ОЦЕНКИ

4.1. Общие критерии для всех траекторий (50%)

Категория	Критерий	Вес
-----------	----------	-----

Аналитика (15%)	Бизнес-анализ (IDEF0, SWOT, стейкхолдеры)	5%
	Требования (Use Case, Domain Model)	5%
	Проектирование БД (ER-диаграмма, 3НФ)	5%
Документация (20%)	Техническое задание	5%
	Руководство пользователя	5%
	Пояснительная записка	5%
	Git-статистика (коммиты, тепловая карта)	5%
Управление (15%)	Git (регулярность, осмысленность)	10%
	Выполнение сроков	5%

4.2. Специфические критерии по траекториям (50%)

Траектория А (50%)

Критерий	Вес	Описание
Модели данных	5%	2+ модели, связи, ForeignKey
Представления (views)	10%	5+ представлений, пагинация
Шаблоны (templates)	10%	Базовый шаблон, наследование, Bootstrap
Сессионная аутентификация	10%	Регистрация, вход, выход, защита
AJAX-запросы	5%	Динамическое обновление контента
Качество интерфейса	5%	Адаптивность, удобство
Дополнительный функционал	5%	Комментарии, избранное

Траектория Б (50%)

Критерий	Вес	Описание
Модели данных	5%	2+ модели, связи, ForeignKey
REST API	10%	8+ эндпоинтов, ViewSet'ы, пагинация
Сериализаторы	5%	Вложенные сериализаторы, валидация
JWT-аутентификация	10%	Токены, refresh, защита маршрутов
React frontend	10%	5+ компонентов, маршрутизация, состояние
Интеграция (CORS, API)	5%	Взаимодействие с API, обработка ошибок
Качество кода	5%	Читаемость, структура, комментарии

Траектория В (50%)

Критерий	Вес	Описание
Модели данных	5%	2+ модели, связи, ForeignKey
REST API	5%	8+ эндпоинтов, ViewSet'ы
JWT-аутентификация	5%	Токены, refresh, защита
React frontend	5%	Компоненты, маршрутизация
Расширенная клиентская логика	10%	Оптимистичные обновления, кэширование
WebSocket (Channels)	10%	Real-time уведомления
Пагинация, фильтрация, поиск	5%	Продвинутая работа с API
Дополнительный функционал	5%	Комментарии, избранное с моментальным отображением

4.3. Итоговая оценка

Траектория	Максимальная оценка	Условие получения
А	4 (хорошо)	Выполнение всех этапов, качественная документация
Б	5 (отлично)	Выполнение всех этапов, качественная документация, работающее SPA
В	5 (отлично) + автомат	Выполнение всех этапов, продвинутая клиентская логика, WebSocket, качественная документация

ЧАСТЬ 5. ТРЕБОВАНИЯ ОФОРМЛЕНИЯ ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ

5.1. Общие требования написания курсового проекта

Курсовая работа должна оформляться в соответствии с требованиями государственных стандартов по инженерным специальностям и направлениям подготовки:

ГОСТ 2.105-95. Общие требования к текстовым документам, ГОСТ 2.106-96. Текстовые документы;

ГОСТ 2.104-68. Основные надписи; Единая система конструкторской документации; Единая система технологической документации; Единая система программной документации;

ГОСТ 7.32-2001. Отчет по научно-исследовательской работе. Структура и правила оформления;

ГОСТ Р 7.0.11–2011. Система стандартов по информации, библиотечному и издательскому делу.

Структура и правила оформления. Содержание курсовой работы рекомендуется представлять в объеме 3-4 печатных листа (примерно 48 – 64 страницы формата А4). Текст пояснительной записки должен быть напечатан через 1,5 интервала на одной стороне стандартного листа белой бумаги (А-4). Текст и другие отпечатанные элементы должны быть черными, контуры букв

и знаков – четкими, без ореола и затенения. Шрифт TimesNewRoman, кегель 14. Курсив и подчеркивание в проекте не допускаются. Названия глав и параграфов выделяются полужирным шрифтом. Лист с текстом должен иметь поля:

слева – 30 мм;
справа – 10 мм;
сверху – 20 мм;
снизу – 20 мм.

Нумерация страниц текста делается в правом нижнем углу листа. Проставлять номер страницы необходимо со страницы, где печатается «Содержание», на которой ставится цифра «4». После этого нумеруются все страницы, включая приложения. Список литературы должен быть составлен в соответствии с требованиями ГОСТа к оформлению библиографии (не менее 25 источников):

ГОСТ 7.1-2003. Библиографическая запись. Библиографическое описание;

ГОСТ 7.82-2001. Библиографическая запись. Библиографическое описание электронных ресурсов;

ГОСТ Р 7.0.5-2008. Библиографическая ссылка. Общие требования и правила составления;

ГОСТ 7.80-2000. Библиографическая запись. Заголовок. Общие требования и правила составления.

Между названием главы и названием параграфа этой главы ставится пробел равный двум интервалам, а название параграфа не должно отделяться от текста этого параграфа пробелом. Названия параграфов отделяются от текста предыдущего параграфа пробелом, равным двум интервалам. Каждая глава, а также введение, выводы, предложения и список использованной литературы начинаются с новой страницы. Слово «Глава» не пишется. Главы имеют порядковые номера в пределах всей работы, обозначаемые арабскими

цифрами (например: 1,2,3), после которых ставится точка. Слово «параграф» или значок параграфа в названии не ставятся. Параграфы имеют порядковые номера в пределах глав, обозначаемые арабскими цифрами (например: 1.1. и 1.2.). Заголовки глав и параграфов в тексте работы должны располагаться по центру, точку в конце названия главы и параграфа не ставят. Не допускается переносить часть слова в заголовке. Нумерация таблиц и рисунков может быть сквозной или соотноситься с номером главы и параграфа. Например, если таблица или рисунок включены в текст первого параграфа второй главы, нумерация следующая: Таблица 2.1.1., рис. 2.1.1. Последняя цифра означает порядковый номер таблицы (или рисунка) в данном параграфе. Таблица помещается в качестве следующей страницы после первого упоминания о ней в тексте. Листы отчета по курсовом проекте должны быть пронумерованы и скреплены.

5.2. Шаблоны содержания пояснительной записки по траекториям

Траектория А: Full-stack Django (классический подход)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

Актуальность

Цель и задачи

Объект и предмет исследования

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

1.2. Анализ бизнес-процессов (IDEF0)

1.3. SWOT-анализ текущего состояния

- 1.4. Анализ аналогов и конкурентов
- 1.5. Обоснование необходимости разработки
- 1.6. Экономическое обоснование (ROI)

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Модель требований к ПО

- 2.1.1. Use Case диаграмма
- 2.1.2. Спецификация прецедентов
- 2.1.3. Глоссарий терминов

2.2. Модель предметной области

- 2.2.1. Domain Model (диаграмма классов)
- 2.2.2. Описание сущностей и атрибутов
- 2.2.3. Бизнес-правила

2.3. Архитектурное проектирование

- 2.3.1. Выбор архитектурного стиля (MTV)
- 2.3.2. Диаграмма компонентов Django
- 2.3.3. Описание слоёв и их ответственности
- 2.3.4. Схема взаимодействия клиент-сервер

2.4. Проектирование базы данных

- 2.4.1. ER-диаграмма
- 2.4.2. Физическая модель данных
- 2.4.3. DDL-скрипты

2.5. Детальное проектирование

- 2.5.1. Диаграммы последовательности (регистрация, создание статьи)
- 2.5.2. Диаграммы классов проектирования
- 2.5.3. Применение паттернов (Repository, Form)

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

3.1. Реализация бизнес-логики

3.1.1. Структура проекта Django

3.1.2. Модели-сущности (Category, Article, User)

3.1.3. Слой доступа к данным (ORM Django)

3.1.4. Слой управления (views, forms)

3.2. Рефакторинг и оптимизация

3.2.1. Статический анализ кода (flake8, pylint)

3.2.2. Применение паттернов (Select_related, Prefetch_related)

3.2.3. Оптимизация запросов к БД

3.3. Пользовательский интерфейс

3.3.1. Django Templates с Bootstrap

3.3.2. Адаптивная вёрстка

3.3.3. AJAX-запросы для динамического обновления

3.4. Безопасность и транзакции

3.4.1. Сессионная аутентификация

3.4.2. Защита от CSRF (csrf_token)

3.4.3. Валидация форм и защита от XSS

3.5. REST API (опционально)

3.5.1. Спецификация API (для AJAX)

3.5.2. Реализация эндпоинтов

3.5.3. Тестирование API

4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

4.1. Модульное тестирование (unittest, pytest)

4.2. Интеграционное тестирование

4.3. Системное тестирование

4.4. Результаты тестирования

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Инструкция по установке (локальный сервер)

5.2. Инструкция по настройке (settings.py, .env)

5.3. Требования к окружению (Python, Django, SQLite)

6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS (иерархическая структура работ)

6.2. Диаграмма Ганта

6.3. Оценка трудозатрат (COCOMO)

6.4. Управление рисками

ЗАКЛЮЧЕНИЕ

Выводы

Результаты

Перспективы развития

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЯ

Приложение А. Листинги кода (models.py, views.py, templates)

Приложение Б. Скриншоты интерфейсов

Приложение В. Результаты тестирования

Траектория Б: Django REST + React SPA (современный подход)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

Актуальность

Цель и задачи

Объект и предмет исследования

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

- 1.1. Описание предметной области
- 1.2. Анализ бизнес-процессов (IDEF0)
- 1.3. SWOT-анализ текущего состояния
- 1.4. Анализ аналогов и конкурентов
- 1.5. Обоснование необходимости разработки
- 1.6. Экономическое обоснование (ROI)

2. ПРОЕКТНАЯ ЧАСТЬ

- 2.1. Модель требований к ПО
 - 2.1.1. Use Case диаграмма
 - 2.1.2. Спецификация прецедентов
 - 2.1.3. Глоссарий терминов
- 2.2. Модель предметной области
 - 2.2.1. Domain Model (диаграмма классов)
 - 2.2.2. Описание сущностей и атрибутов
 - 2.2.3. Бизнес-правила
- 2.3. Архитектурное проектирование
 - 2.3.1. Выбор архитектурного стиля (клиент-сервер)
 - 2.3.2. Диаграмма компонентов (Django + React)
 - 2.3.3. Описание слоёв и их ответственности
 - 2.3.4. Схема взаимодействия (REST API + JWT)
- 2.4. Проектирование базы данных
 - 2.4.1. ER-диаграмма
 - 2.4.2. Физическая модель данных
 - 2.4.3. DDL-скрипты
- 2.5. Детальное проектирование
 - 2.5.1. Диаграммы последовательности (аутентификация, CRUD)
 - 2.5.2. Диаграммы классов проектирования (Django + React)
 - 2.5.3. Применение паттернов (ViewSet, Serializer, React Hooks)

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

3.1. Реализация бизнес-логики (Backend)

3.1.1. Структура проекта Django

3.1.2. Модели-сущности (Category, Article, User)

3.1.3. Слой доступа к данным (ORM Django)

3.1.4. Слой управления (ViewSet, Serializer)

3.2. Реализация API

3.2.1. REST API эндпоинты (8+)

3.2.2. JWT-аутентификация (access/refresh токены)

3.2.3. Разрешения (permissions) для разграничения доступа

3.2.4. Документация API (Swagger/OpenAPI)

3.3. Реализация фронтенда (React)

3.3.1. Структура проекта React

3.3.2. Компоненты (ArticleList, ArticleDetail, Login, Register)

3.3.3. Маршрутизация (React Router)

3.3.4. Управление состоянием (AuthContext)

3.3.5. Взаимодействие с API (Axios, interceptors)

3.4. Рефакторинг и оптимизация

3.4.1. Статический анализ кода (flake8, ESLint)

3.4.2. Оптимизация запросов (select_related, prefetch_related)

3.4.3. Кэширование на клиенте (React Query / RTK Query)

3.5. Безопасность и транзакции

3.5.1. JWT-аутентификация

3.5.2. Защита от XSS, CSRF (CORS настройки)

3.5.3. Валидация данных на сервере и клиенте

4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

4.1. Модульное тестирование бэкенда (pytest, APITestCase)

4.2. Модульное тестирование фронтенда (Jest, React Testing Library)

4.3. Интеграционное тестирование (бэкенд + фронтенд)

4.4. Результаты тестирования

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Контейнеризация (Docker)

5.1.1. Dockerfile для Django

5.1.2. Dockerfile для React

5.1.3. docker-compose.yml

5.2. Инструкция по установке

5.3. Инструкция по настройке (.env, CORS)

5.4. Требования к окружению (Python, Node.js, PostgreSQL)

6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS (иерархическая структура работ)

6.2. Диаграмма Ганта

6.3. Оценка трудозатрат (COCOMO)

6.4. Управление рисками

ЗАКЛЮЧЕНИЕ

Выводы

Результаты

Перспективы развития

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЯ

Приложение А. Листинги кода (models.py, serializers.py, React components)

Приложение Б. Скриншоты интерфейсов

Приложение В. Результаты тестирования

Приложение Г. Документация API (OpenAPI)

Траектория В: React SPA + AJAX + JWT (расширенный подход)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

Актуальность

Цель и задачи

Объект и предмет исследования

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

1.2. Анализ бизнес-процессов (IDEF0)

1.3. SWOT-анализ текущего состояния

1.4. Анализ аналогов и конкурентов

1.5. Обоснование необходимости разработки

1.6. Экономическое обоснование (ROI)

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Модель требований к ПО

2.1.1. Use Case диаграмма

2.1.2. Спецификация прецедентов

2.1.3. Глоссарий терминов

2.2. Модель предметной области

2.2.1. Domain Model (диаграмма классов)

2.2.2. Описание сущностей и атрибутов

2.2.3. Бизнес-правила

2.3. Архитектурное проектирование

- 2.3.1. Выбор архитектурного стиля (клиент-сервер + WebSocket)
- 2.3.2. Диаграмма компонентов (Django + React + Channels)
- 2.3.3. Описание слоёв и их ответственности
- 2.3.4. Схема взаимодействия (REST API + WebSocket + JWT)
- 2.4. Проектирование базы данных
 - 2.4.1. ER-диаграмма
 - 2.4.2. Физическая модель данных
 - 2.4.3. DDL-скрипты
- 2.5. Детальное проектирование
 - 2.5.1. Диаграммы последовательности (аутентификация, CRUD, WebSocket)
 - 2.5.2. Диаграммы классов проектирования (Django + React)
 - 2.5.3. Применение паттернов (ViewSet, Serializer, React Hooks, Observer)

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

- 3.1. Реализация бизнес-логики (Backend)
 - 3.1.1. Структура проекта Django
 - 3.1.2. Модели-сущности (Category, Article, Comment, User)
 - 3.1.3. Слой доступа к данным (ORM Django)
 - 3.1.4. Слой управления (ViewSet, Serializer)
- 3.2. Реализация API
 - 3.2.1. REST API эндпоинты (10+)
 - 3.2.2. JWT-аутентификация (access/refresh токены)
 - 3.2.3. Разрешения (permissions) для разграничения доступа
 - 3.2.4. Документация API (Swagger/OpenAPI)
- 3.3. Реализация WebSocket (Django Channels)
 - 3.3.1. Настройка ASGI и Channel Layers
 - 3.3.2. WebSocket Consumer (комментарии в реальном времени)
 - 3.3.3. Маршрутизация WebSocket

3.4. Реализация фронтенда (React)

3.4.1. Структура проекта React

3.4.2. Компоненты (ArticleList, ArticleDetail, Comments, Login)

3.4.3. Маршрутизация (React Router)

3.4.4. Управление состоянием (AuthContext, Redux Toolkit)

3.4.5. Взаимодействие с API (Axios, interceptors)

3.4.6. WebSocket клиент (useWebSocket хук)

3.5. Расширенная клиентская логика

3.5.1. Оптимистичные обновления (комментарии, избранное)

3.5.2. Кэширование запросов (React Query / RTK Query)

3.5.3. Пагинация, фильтрация, поиск на клиенте

3.5.4. Real-time уведомления (WebSocket)

3.6. Рефакторинг и оптимизация

3.6.1. Статический анализ кода (flake8, ESLint, Prettier)

3.6.2. Оптимизация запросов (select_related, prefetch_related)

3.6.3. Оптимизация React (React.memo, useMemo, useCallback)

3.7. Безопасность и транзакции

3.7.1. JWT-аутентификация с refresh токенами

3.7.2. Защита от XSS, CSRF (CORS настройки)

3.7.3. Валидация данных на сервере и клиенте

3.7.4. Аутентификация WebSocket (AuthMiddlewareStack)

4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

4.1. Модульное тестирование бэкенда (pytest, APITestCase)

4.2. Модульное тестирование фронтенда (Jest, React Testing Library)

4.3. Интеграционное тестирование

4.4. Нагрузочное тестирование WebSocket

4.5. Результаты тестирования

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Контейнеризация (Docker)

5.1.1. Dockerfile для Django

5.1.2. Dockerfile для React

5.1.3. Dockerfile для Redis (Channel Layers)

5.1.4. docker-compose.yml (Django, React, PostgreSQL, Redis)

5.2. Инструкция по установке

5.3. Инструкция по настройке (.env, CORS, Channel Layers)

5.4. Требования к окружению (Python, Node.js, PostgreSQL, Redis)

6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS (иерархическая структура работ)

6.2. Диаграмма Ганта

6.3. Оценка трудозатрат (COCOMO)

6.4. Управление рисками

ЗАКЛЮЧЕНИЕ

Выводы

Результаты

Перспективы развития

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЯ

Приложение А. Листинги кода (models.py, serializers.py, consumers.py, React components)

Приложение Б. Скриншоты интерфейсов

Приложение В. Результаты тестирования

Приложение Г. Документация API (OpenAPI)

Приложение Д. WebSocket протокол

Список рекомендуемой литературы

10. Учебно-методическое и информационное обеспечение дисциплины

10.1. Перечень основной и дополнительной литературы, необходимой для освоения дисциплины (модуля)

10.1.1. Перечень основной литературы:

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. – Санкт-Петербург : Питер, 2025. – 496 с. – (Библиотека программиста). – ISBN 978-5-4461-4272-9 -7-9-10.
2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. – Санкт-Петербург : Наука и Техника, 2026. – 512 с. – ISBN 978-5-907592-84-1 -3.
3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. – Санкт-Петербург : Питер, 2025. – 384 с. – (Библиотека программиста). – ISBN 978-5-4461-4201-9

10.1.2. Перечень дополнительной литературы:

1. Курс «Фронтенд-разработчик» от Яндекс Практикум <https://practicum.yandex.ru/frontend-developer/>
2. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5
3. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров, С. П. Ильин, П. А. Гершевич. — Санкт-Петербург : Питер, 2026. — 688 с. — (Библиотека программиста). — ISBN 978-5-4461-4469-3

10.2. Перечень ресурсов информационно-телекоммуникационной сети «Интернет», необходимых для освоения дисциплины (модуля):

1. <https://metanit.com/>

Приложение 1

Министерство науки и высшего образования и Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Кафедра межинститутская базовая

Направление подготовки 09.03.04 «Программная инженерия»

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

ЗАДАНИЕ

на курсовой проект

студента Иванова Ивана Ивановича

(фамилия, имя, отчество)

по дисциплине «Технология разработки программного обеспечения»

1. Тема проекта Тема КП в соответствии с приказом

2. Цель: Повышение уровня профессиональной подготовки путем углубления и закрепления теоретических знаний и практических навыков, приобретенных в результате изучения дисциплины «Технология разработки программного обеспечения», а также формирование компетенций в области проектирования, разработки и документирования современных веб-приложений с использованием фреймворка Django в рамках выбранной траектории выполнения.

3. Задачи:

3.1. Анализ прикладной задачи и методов её решения

- Провести предпроектное обследование предметной области.
- Выполнить анализ бизнес-процессов и сформулировать требования к программному обеспечению.
- Обосновать выбор технологического стека в соответствии с выбранной траекторией: *(записать свою выбранную траекторию)*
 - Траектория А: Django, Django Templates, Bootstrap, SQLite/PostgreSQL.
 - Траектория Б: Django, Django REST Framework, React, JWT, PostgreSQL.
 - Траектория В: Django, Django REST Framework, React, Django Channels, WebSocket, JWT, PostgreSQL, Redis.
- Проанализировать существующие аналоги и обосновать необходимость разработки.

3.2. Разработка алгоритмов решения задачи

- Разработать модель предметной области (Domain Model).
- Спроектировать Use Case диаграмму и спецификации прецедентов.
- Разработать ER-диаграмму базы данных.
- Спроектировать архитектуру приложения в соответствии с выбранной траекторией: *(записать свою выбранную траекторию)*
 - Траектория А: архитектура Django MTV.
 - Траектория Б: клиент-серверная архитектура с REST API.
 - Траектория В: клиент-серверная архитектура с WebSocket.
- Разработать алгоритмы основных сценариев (регистрация, аутентификация, CRUD-операции).

3.3. Реализация программного кода

- Реализовать модели данных с учётом связей между таблицами.
- Реализовать систему аутентификации: *(записать свою выбранную траекторию)*
 - Траектория А: сессионная аутентификация.
 - Траектория Б, В: JWT-аутентификация (access/refresh токены).
- Реализовать бизнес-логику:
 - Траектория А: представления (views) и шаблоны (templates).
 - Траектория Б, В: REST API (ViewSet, Serializer) и React-приложение.
- Реализовать дополнительные функции согласно выбранной траектории:
 - Траектория А: AJAX-запросы для динамического обновления контента.
 - Траектория Б: полноценное SPA с маршрутизацией и управлением состоянием.
 - Траектория В: WebSocket (Django Channels) для real-time уведомлений, оптимистичные обновления, кэширование.

3.4. Отладка и тестирование программного кода

- Провести модульное тестирование ключевых компонентов (модели, API, компоненты React).
- Выполнить интеграционное тестирование взаимодействия между слоями.

- Провести системное тестирование сценариев использования.
- Для траектории В: выполнить нагрузочное тестирование WebSocket-соединений.
- Зафиксировать результаты тестирования в отчёте.

4. Перечень подлежащих разработке вопросов:

- а) теоретической части: изучение и анализ литературы, постановка условия задачи, выбор и описание методов и библиотек для ее решения и технологий, среды программирования
- б) проектная часть: проектирование UML-диаграмм классов, разработка алгоритмов решения задачи, методов классов
- в) реализация: описание структуры проекта, описание файлов проекта, разработка программного кода, тестирование программы

5. Исходные данные:

- а) по литературным источникам: ГОСТы, международные стандарты, программные средства, используемые при разработке программного обеспечения
- б) исходные данные, подготовленные для тестирования объекта профессиональной деятельности (информационной системы/приложения/программного продукта)
- 6. Список рекомендуемой литературы монографии, диссертации, научные статьи, учебно-методические материалы, ссылки на официальные сайты, содержащие информацию необходимую для решения поставленных в работе задач *(перечислить не менее 25 литературных источников, которые применялись при выполнении КР)*

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. – Санкт-Петербург : Питер, 2025. – 496 с. – (Библиотека программиста). – ISBN 978-5-4461-4272-9 -7-9-10.

2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. – Санкт-Петербург : Наука и Техника, 2026. – 512 с. – ISBN 978-5-907592-84-1 -3.

3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. – Санкт-Петербург : Питер, 2025. – 384 с. – (Библиотека программиста). – ISBN 978-5-4461-4201-9

4. Курс «Фронтенд-разработчик» от Яндекс Практикум
<https://practicum.yandex.ru/frontend-developer/>

5. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5

6. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров, С. П. Ильин, П. А. Гершевич. — Санкт-Петербург : Питер, 2026. — 688 с. — (Библиотека программиста). — ISBN 978-5-4461-4469-3

7. Контрольные сроки представления отдельных разделов курсовой работы:

- | | |
|--|---------------------|
| 25 % - предоставление первого раздела | «12» марта 2026 г. |
| 50 % - предоставление второго раздела | «09» апреля 2026 г. |
| 75 % - предоставление третьего раздела | «30» апреля 2026 г. |
| 100 % - предоставление работы на отзыв | «14» мая 2026 г. |

8. Срок защиты студентом курсовой работы «23» мая 2026 г.

Дата выдачи задания «12» февраля 2026 г.

Руководитель курсового проекта

К.т.н.

(ученая степень, звание)

(личная подпись)

Самойлов Ф.В.

(инициалы, фамилия)

Задание принял к исполнению студент очной формы обучения 2 курса
группы ПИЖ-б-о-24-1

(личная подпись)

И.И. Иванов

(инициалы, фамилия)

Приложение 2

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ЦИФРОВОГО РАЗВИТИЯ
МЕЖИНСТИТУТСКАЯ БАЗОВАЯ КАФЕДРА

КУРСОВОЙ ПРОЕКТ

по дисциплине
«Технология разработки программного обеспечения»
на тему:

« _____ »

Выполнил:

Иванов Иван Иванович

студент 2 курса

группы ПИЖ-б-о-24-1

направления подготовки 09.03.04

«Программная инженерия»

направленность (профиль)

«Разработка и сопровождение
программного обеспечения»

очной формы обучения

(подпись)

Руководитель работы:

Ф.В. Самойлов, доцент

межинститутской базовой кафедры

Работа допущена к защите

(подпись руководителя)

(дата)

Работа выполнена и
защищена с оценкой

Дата защиты _____

Председатель комиссии:

зав. межинститутской

базовой кафедрой

(подпись)

Е. Н. Новикова

Члены комиссии:

доцент МИБК

(подпись)

Ф.В. Самойлов

доцент МИБК

(подпись)

И.В. Свистунов

Ставрополь, 2026 г.